

Graphics Programming

Final Commentary

Name: Dhanarasu Naveen

SIM Student ID: 10250211

UOL Student ID: 230655533

GP - Commentary

In this project, I explored several image processing techniques using a live webcam feed, focusing on image thresholding across individual color channels and alternative color spaces. When processing the red channel, I observed that thresholding produced results different from those of the green and blue channels. The red channel often carried higher intensity values, requiring careful calibration to set an appropriate threshold. Often, red exhibited stronger contrast between high and low intensity regions, making the thresholded image appear less noisy and more defined. However, in areas dominated by red, thresholding sometimes led to a loss of subtle detail, masking gradual intensity changes.

A significant challenge was ensuring that, after increasing brightness by 20% during grayscale conversion, no pixel value exceeded 255. Exceeding this limit would have caused artifacts and distortions. To address this, I implemented a boundary check within the same loop used for grayscale conversion. This straightforward solution brightened the image while clipping pixel values, preserving quality without unwanted effects.

Another technical hurdle was integrating real-time face detection and applying filters to detected faces. Occasionally, while processing the red channel and applying thresholding, synchronization issues arose between live video capture and subsequent processing routines. I resolved these problems by optimizing the processing loop and ensuring pixel operations were efficient. By refining the code structure and using object-oriented design, I isolated computationally intensive tasks, reducing latency and improving responsiveness.

Beyond RGB processing, I performed color space conversions using HSV and YCbCr algorithms. These alternative representations provided improved segmentation results in thresholding. I found that while thresholding in RGB sometimes produced noisy images—especially in the green and blue channels—applying thresholding after converting to HSV or YCbCr yielded cleaner, more defined results. This likely stems from these color spaces separating luminance from chrominance, allowing for more precise feature isolation without interference from color noise.

To further enrich the project, I implemented six distinct extensions that add creative, interactive dimensions. The first extension introduces a Glitch/Cyberpunk effect, creating a futuristic look by randomly offsetting pixel channels. The second extension involves Hand Gesture Filters, which apply neon or x-ray effects based on detected hand movements. The third extension is a Blink Reaction Game that challenges users to blink in response to prompts while measuring reaction times. The fourth extension implements Snapchat-Style Filters that overlay fun accessories—such as hats, glasses, or masks—onto detected faces. The fifth extension transforms the live feed into a Single-Paddle Pong game, with head movement tracked by PoseNet controlling the paddle.

The primary and most innovative extension is the sixth: Hand Keypoints and Emoji Overlay. This extension leverages the ml5.js handpose model to track hand movements and detect gestures by analyzing key landmarks like fingertips and the palm center. Based on which fingers are extended, the system dynamically computes and overlays an appropriate emoji on the display. Calibration was challenging due to variability in hand positions, so I introduced a smoothing algorithm that averages recent hand poses, resulting in more stable and reliable gesture recognition.